

Computação e Computacional Pensamento

Alfred V. Aho

Departamento de Ciência da Computação Columbia University Nova York, NY 10027

Autor correspondente: aho@cs.columbia.edu

Recomendamos usar o termo Computação em conjunto com um modelo de computação bem definido, cuja semântica seja clara e que corresponda ao problema que está sendo investigado. A ciência da computação já possui uma série de modelos de computação úteis e claramente definidos, cujos comportamentos e capacidades são bem compreendidos. Deveríamos usar tais modelos como parte de qualquer definição do termo computação. Contudo, para novos domínios de investigação onde não existam modelos apropriados, poderá ser necessário inventar novos formalismos para representar os sistemas em estudo.

1. A NECESSIDADE DE DEFINIÇÕES CLARAS

Em qualquer disciplina científica há muitas razões para usar termos que tenham definições precisas. Compreender a terminologia de uma disciplina é essencial para aprender um assunto e uma terminologia precisa nos permite comunicar ideias claramente com outras pessoas. Na ciência da computação o problema é ainda mais grave: precisamos construir componentes de software e hardware que interoperem suavemente através de interfaces com clientes e outros componentes em sistemas distribuídos. As definições dessas interfaces precisam ser especificadas com precisão para garantir a interoperabilidade e o bom desempenho dos sistemas.

Usar o termo “computação” sem qualificação muitas vezes gera muita confusão. Parte do problema é que a natureza dos sistemas que exibem comportamento computacional é variada e o termo computação significa coisas diferentes para pessoas diferentes, dependendo dos tipos de sistemas computacionais que estão estudando e dos tipos de problemas que estão investigando.

Como a computação se refere a um processo definido em termos de algum modelo subjacente de computação, conseguiríamos uma comunicação mais clara se esclarecêssemos qual é o modelo subjacente.

Em vez de falar sobre uma noção vaga de “computação”, minha sugestão é usar o termo em conjunto com um modelo de computação bem definido, cuja semântica seja clara e que corresponda ao problema que está sendo investigado. A ciência da computação já possui uma série de modelos de computação úteis e claramente definidos, cujos comportamentos e capacidades são bem compreendidos. Deveríamos usar tais modelos como parte de qualquer definição do termo computação. Contudo, para novos domínios de investigação onde não existam modelos apropriados, poderá

será necessário inventar novos formalismos para representar os sistemas em estudo.

2. PENSAMENTO COMPUTACIONAL

Consideramos o pensamento computacional os processos de pensamento envolvidos na formulação de problemas para que suas soluções possam ser representadas como etapas computacionais e algoritmos. Uma parte importante deste processo é encontrar modelos de computação apropriados para formular o problema e derivar suas soluções. Um exemplo familiar seria o uso de autômatos finitos para resolver problemas de correspondência de padrões de strings. Um exemplo menos familiar pode ser os circuitos quânticos e a formulação de determinação de ordem que Peter Shor [1] usou para desenvolver um algoritmo de fatoração de números inteiros que roda em tempo polinomial em um computador quântico. Associada aos modelos básicos de computação na ciência da computação está uma riqueza de técnicas bem conhecidas de design de algoritmos e resolução de problemas que podem ser usadas para resolver problemas comuns que surgem na computação.

No entanto, à medida que os sistemas informáticos que desejamos construir se tornam mais complexos e à medida que aplicamos abstrações da ciência da computação a novos domínios de problemas, descobrimos que nem sempre temos os modelos apropriados para conceber soluções. Nestes casos, o pensamento computacional torna-se uma atividade de pesquisa que inclui a invenção de novos modelos de computação apropriados.

Corrado Priami e seus colegas do Centro de Biologia Computacional e de Sistemas em Trento, Itália, têm usado cálculos de processos como modelo de computação para criar linguagens de programação para simular processos biológicos.

Priami afirma que “a característica básica do pensamento computacional é

abstração da realidade de tal forma que os detalhes negligenciados no modelo o tornem executável por uma máquina". [2]

Como veremos, encontrar ou conceber modelos de computação apropriados para formular problemas é uma parte central e muitas vezes não trivial do pensamento computacional.

3. FORÇAS EM JOGO

No último meio século, o que consideramos um sistema computacional expandiu-se dramaticamente. Nos primórdios da computação, um computador era uma máquina isolada com memória limitada à qual os programas eram submetidos, um de cada vez, para serem compilados e executados. Hoje, na era da Internet, temos redes que consistem em milhões de computadores interligados e, à medida que avançamos para a computação em nuvem, muitos prevêem um ambiente de computação global com milhares de milhões de clientes com acesso universal a pedido a serviços de computação e dados alojados em centros de dados gigantescos. localizados ao redor do planeta. Qualquer coisa de um PC, telefone, TV ou sensor pode ser um cliente e um data center pode consistir em centenas de milhares de servidores. Escusado será dizer que os modelos para estudar um sistema distribuído tão universalmente acessível, complexo e altamente concorrente são muito diferentes daqueles para um único computador isolado.

Outra força em jogo é que, devido a considerações de dissipação de calor, a arquitetura dos computadores está mudando. Um PC comum hoje tem muitos elementos de computação diferentes, como chips multicore e unidades de processamento gráfico, e espera-se que um supercomputador em exaescala no final desta década seja uma máquina paralela gigante com até um milhão de nós, cada um com possivelmente mil processadores. Nossa compreensão de como escrever programas eficientes para essas máquinas é limitada. Bons modelos de computação paralela e técnicas de projeto de algoritmos paralelos são uma área de pesquisa vital e aberta para uma computação paralela eficaz.

Além disso, há um interesse crescente em aplicar a computação ao estudo de praticamente todas as áreas da atividade humana. Um exemplo fascinante é a simulação de processos biológicos altamente paralelos encontrados em células e órgãos humanos para fins de compreensão de doenças e design de medicamentos. Bons modelos computacionais para processos biológicos ainda estão engatinhando. E não está claro se algum dia seremos capazes de encontrar um modelo computacional para o cérebro humano que dê conta de fenômenos emergentes como a consciência ou a inteligência.

4. A TEORIA DA COMPUTAÇÃO

A teoria da computação foi e ainda é uma das áreas centrais da ciência da computação. Ele explora as capacidades e limitações fundamentais dos modelos de computação. Um modelo de computação é uma abstração matemática de um sistema de computação. O modelo mais importante de computação sequencial estudado em ciência da computação é a máquina de Turing, primeiro

proposto por Alan Turing em 1936 [3]. Vamos revisar brevemente a definição de máquina de Turing para apreciar os detalhes necessários para compreender até mesmo esse modelo familiar de computação.

Podemos pensar em uma máquina de Turing como um controle de estado finito conectado a um cabeçote de fita que pode ler e escrever símbolos nos quadrados de uma fita semi-infinita. Inicialmente, uma string finita de comprimento n representando a entrada está nos n quadrados mais à esquerda da fita. Uma sequência infinita de espaços em branco segue a string de entrada. A cabeça da fita está lendo o símbolo no quadrado mais à esquerda e o controle finito está em um estado inicial predefinido.

A máquina de Turing então faz uma sequência de movimentos. Em um movimento, ele lê o símbolo na fita sob o cabeçote da fita e consulta uma tabela de transição no controle de estado finito que especifica um símbolo a ser impresso no quadrado sob o cabeçote da fita, uma direção em que o cabeçote da fita deve se mover (uma quadrado para a esquerda ou direita) e um estado para entrar em seguida. Se a máquina de Turing entrar em um estado de parada de aceitação (um sem próximo movimento), a sequência de símbolos não vazios restantes na fita de entrada naquele momento será sua saída.

Matematicamente, uma máquina de Turing consiste em sete componentes: um conjunto finito de estados; um alfabeto de entrada finito (sem espaço em branco); um alfabeto de fita finito (que inclui o alfabeto de entrada e o espaço em branco); uma função de transição que mapeia um estado e um símbolo de fita em um estado, símbolo de fita e direção (esquerda ou direita); um estado inicial; um estado de aceitação a partir do qual não há mais movimentos; e um estado de rejeição a partir do qual não há mais movimentos.

Podemos caracterizar a configuração de uma máquina de Turing em um determinado momento por três quantidades:

- (i) o estado do controle de estado finito, (ii) a sequência de símbolos não vazios na fita e (iii) a localização do cabeçote de entrada na fita.

Um cálculo de uma máquina de Turing em uma entrada w é uma sequência de configurações pelas quais a máquina pode passar, começando pela configuração inicial com w na fita e terminando (se o cálculo terminar) em uma configuração de parada. Dizemos que uma função f de string para string é computável se houver alguma máquina de Turing M que, dada qualquer string de entrada w , sempre pare no estado de aceitação com apenas $f(w)$ em sua fita. Dizemos que M calcula f .

A máquina de Turing fornece uma definição precisa para o termo algoritmo: um algoritmo para uma função f é apenas uma máquina de Turing que calcula f .

Existem dezenas de modelos de computação que são equivalentes às máquinas de Turing, no sentido de que esses modelos computam exatamente o mesmo conjunto de funções que as máquinas de Turing podem calcular. Entre esses modelos de computação completos de Turing estão máquinas de Turing multifita, cálculo lambda, cálculo aleatório acessar máquinas, sistemas de produção, autômatos celulares e todas as linguagens de programação de uso geral.

A razão pela qual existem tantos modelos diferentes de computação equivalentes às máquinas de Turing é que raramente queremos implementar um algoritmo como um programa de máquina de Turing;

gostaríamos de usar uma notação computacional, como uma linguagem de programação, que seja fácil de escrever e fácil de entender. Mas não importa a notação que escolhermos, a famosa tese de Church-Turing levanta a hipótese de que qualquer função que possa ser computada pode ser computada por uma máquina de Turing.

Observe que se existe um algoritmo para calcular uma função f , então existe um número infinito. Grande parte da ciência da computação é dedicada a encontrar algoritmos eficientes para calcular uma determinada função.

Para maior clareza, devemos salientar que definimos um cálculo como uma sequência de configurações que uma máquina de Turing pode realizar em uma determinada entrada. Esta sequência pode ser infinita se a máquina não parar ou uma de uma série de sequências possíveis caso a máquina seja não determinística.

A razão pela qual passamos por esta explicação é apontar quantos detalhes estão envolvidos na definição precisa do termo computação para a máquina de Turing, um dos modelos mais simples de computação. Não é surpreendente, portanto, que à medida que avançamos para modelos mais complexos, a quantidade de esforço necessária para formular com precisão a computação em termos desses modelos cresça substancialmente.

5. MODELOS CONCORRENTES

Muitos sistemas computacionais do mundo real computam mais do que apenas uma única função – o mundo mudou para a computação interativa [4]. O termo sistema reativo é usado para descrever um sistema que mantém uma interação contínua com seu ambiente. Exemplos de sistemas reativos incluem sistemas operacionais e sistemas embarcados.

Um sistema distribuído é aquele que consiste em sistemas de computação autônomos que se comunicam entre si através de algum tipo de rede usando passagem de mensagens. Exemplos de sistemas distribuídos incluem sistemas de telecomunicações, Internet, sistemas de controle de tráfego aéreo e computadores paralelos.

Muitos sistemas distribuídos também são sistemas reativos.

Talvez os exemplos mais intrigantes de sistemas de computação distribuídos reativos sejam os sistemas biológicos, como células e organismos. Poderíamos até considerar o cérebro humano um sistema de computação biológica. A formulação de modelos de computação apropriados para a compreensão dos processos biológicos é um formidável desafio científico na interseção da biologia e da ciência da computação.

Sistemas distribuídos podem exibir comportamentos como deadlock, livelock, condições de corrida e similares que não podem ser estudados de forma útil usando um modelo sequencial de computação. Além disso, resolver problemas como determinar o rendimento, a latência e o desempenho de um sistema distribuído não pode ser formulado de forma produtiva com um modelo de computação de thread único. Por estas razões, os cientistas da computação desenvolveram uma série de modelos de computação concorrente que podem ser usados para estudar esses fenômenos e para arquitetar ferramentas e componentes para a construção de sistemas distribuídos.

Existem muitos modelos teóricos para computação concorrente. Um deles é o modelo Ator de passagem de mensagens, que consiste em entidades computacionais chamadas atores [5].

Um ator pode enviar e receber mensagens, tomar decisões locais, criar mais atores e fixar o comportamento a ser usado na próxima mensagem que receber. Estas ações podem ser executadas em paralelo e sem uma ordem fixa. O modelo Ator foi desenvolvido para estudar as propriedades comportamentais de máquinas de computação paralela que consistem em um grande número de processadores independentes que se comunicam através da passagem de mensagens através de uma rede. Outros modelos bem estudados de computação concorrente incluem redes de Petri e cálculos de processo, como cálculo pi e cálculo mu.

Muitas variantes de modelos computacionais para sistemas distribuídos estão sendo desenvolvidas para estudar e compreender o comportamento de sistemas biológicos. Por exemplo, Dematte *et al.* [6] descrevem uma linguagem chamada BlenX que é baseada em um cálculo de processo chamado Beta-binders para modelagem e simulação de sistemas biológicos.

Não temos espaço para descrever esses modelos concorrentes em detalhes. No entanto, ainda é uma área de pesquisa aberta encontrar modelos de computação concorrentes úteis e práticos que combinem controle e dados para muitas áreas da computação distribuída.

6. BENEFÍCIOS DOS MODELOS DE COMPUTAÇÃO

Além de ajudar na educação e na compreensão, há muitos benefícios práticos em ter modelos de computação apropriados para os sistemas que estamos tentando construir. Na computação em nuvem, por exemplo, ainda há uma série de preocupações mal compreendidas para sistemas desta escala. Precisamos compreender melhor as compensações arquitetônicas necessárias para alcançar os níveis desejados de confiabilidade, desempenho, escalabilidade e adaptabilidade nos serviços que se espera que esses sistemas forneçam.

Não temos abstrações apropriadas para descrever essas propriedades de tal forma que possam ser automaticamente mapeadas de um modelo de computação para uma implementação (ou vice-versa).

Na computação em nuvem, há uma série de desafios de pesquisa para desenvolvedores de sistemas e criadores de ferramentas. Como exemplos, precisamos de linguagens de programação, compiladores, ferramentas de verificação, ferramentas de detecção de defeitos e ferramentas de gerenciamento de serviços que possam ser dimensionadas para o grande número de clientes e servidores envolvidos nas redes e data centers do futuro. A computação em nuvem é uma área importante que pode se beneficiar do pensamento computacional inovador.

7. CONCLUSÃO

Abstrações matemáticas chamadas modelos de computação estão no cerne da computação e do pensamento computacional.

A computação é um processo definido em termos de um modelo subjacente de computação e o pensamento computacional são os processos de pensamento envolvidos na formulação de problemas de forma que

suas soluções podem ser representadas como etapas computacionais e algoritmos. Modelos úteis de computação para resolver problemas que surgem na computação sequencial podem variar de máquinas simples de estado finito a modelos completos de Turing, como máquinas de acesso aleatório. Modelos úteis de computação concorrente para resolver problemas que surgem no projeto e análise de sistemas distribuídos complexos ainda são objeto de pesquisas atuais.

SOBRE O AUTOR

Alfred V. Aho é professor Lawrence Gussman no Departamento de Ciência da Computação da Universidade de Columbia. Ele atuou como presidente do departamento de 1995 a 1997 e na primavera de 2003.

RECONHECIMENTOS

O autor gostaria de agradecer a Peter Denning e Jeannette Wing por seus comentários atenciosos sobre a importância do pensamento computacional. O autor também agradece a Jim Larus por seus insights sobre os problemas enfrentados pela computação em nuvem e a Corrado Priami pelas muitas conversas estimulantes sobre pensamento computacional em biologia.

REFERÊNCIAS

- [1] Shor, PW (1994) Algoritmos para computação quântica: logaritmos discretos e fatoração. *Anais do 35º Simpósio Anual sobre Fundamentos da Ciência da Computação*, pp. Sociedade de Computação IEEE.
- [2] Priami, C. (2007) Pensamento computacional em biologia. *Transações em Biologia de Sistemas Computacionais VIII*, 8, 63–76.
- [3] Turing, A. (1937) Sobre números computáveis, com uma aplicação ao problema de entscheidung. *Anais da London Mathematical Society*, 2, **42**, pp.
- [4] Goldin, DQ, Smolka, SA e Wegner, P. (eds.) (2006) *Computação Interativa: O Novo Paradigma*. Springer.
- [5] Hewitt, C., Bishop, P., e Steiger, R. (1973) Um formalismo de ator modular universal para inteligência artificial. *Anais da 3ª conferência conjunta internacional sobre inteligência artificial*, Stanford, EUA, pp. Morgan Kaufmann Publishers Inc.
- [6] Dematté, L., Priami, C. e Romanel, A. (2008) A linguagem blnx: um tutorial. *Anais do SFM'08*, Bertinoro, Itália, pp. Springer-Verlag.
- [7] Denning, PJ (2009) Além do pensamento computacional. *Comum. ACM*, **52**, 28–30.
- [8] Wing, JM (2006) Pensamento computacional. *Comum. ACM*, **49**, 33–35.